**Course:** [A25] Data Pipeline Part 1

# Project Title: FC Méditerranée

**Institution:** DSTI
**Instructor:** Professor Catherine Faron

**Group Members:**

Asma Dridi
Shayma Ridene
Wassim Elmoufakkir
Abdellahi Abdellahi

**Date:** 2026-05-10

# Contents

# 1 Task Distribution

The project workload was distributed equally among the four team members, with each member contributing approximately 25%.

## Asma

- Developed the Python program for XML parsing, XSD validation, and XSLT execution
- Implemented the Java version of the processing pipeline as an optional bonus
- Created two XML-to-XML XSLT transformations:
  - Club players report
  - Training calendar export
- Added comments and documentation to the code

## Abdelahi

- Developed six XSLT stylesheets for XML-to-HTML transformations
- Implemented six HTML visualization scenarios
- Generated and saved all HTML output files
- Documented each XSLT stylesheet

## Shayma

- Developed two XML-to-JSON XSLT transformations
- Created the JSON Schema for output validation
- Tested and reviewed generated files
- Wrote report sections related to:
  - implemented scenarios
  - task distribution
  - AI tools usage

## Wassim

- Designed the XML Schema (XSD)
- Created the source XML dataset
- Wrote report sections related to:
  - modeling choices
  - tools used
- Assembled and packaged the final project deliverables

# 2 Implemented Scenarios

The project includes multiple transformation scenarios implemented by different team members.

## 2.1 XML to XML Scenarios

Two XML transformation scenarios were implemented:

- Player club report generation: transforms the original football club XML into a simplified XML report focused on players and statistics.

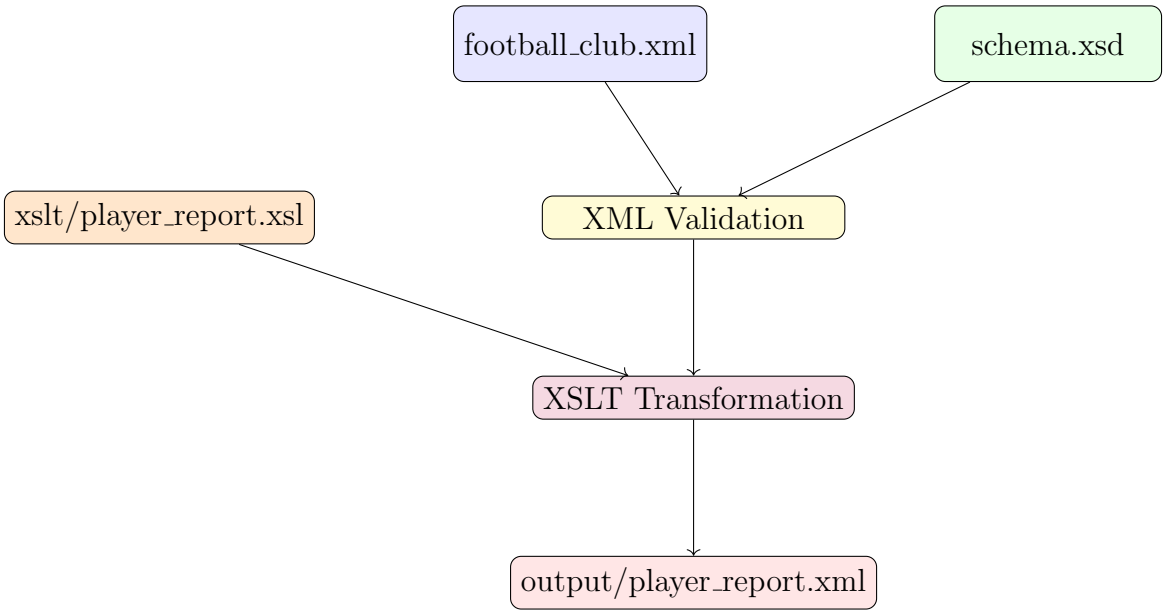


Figure 1: Player Club Report Generation (XML → XML)

- Training calendar generation: transforms training session data into a structured calendar-oriented XML document.

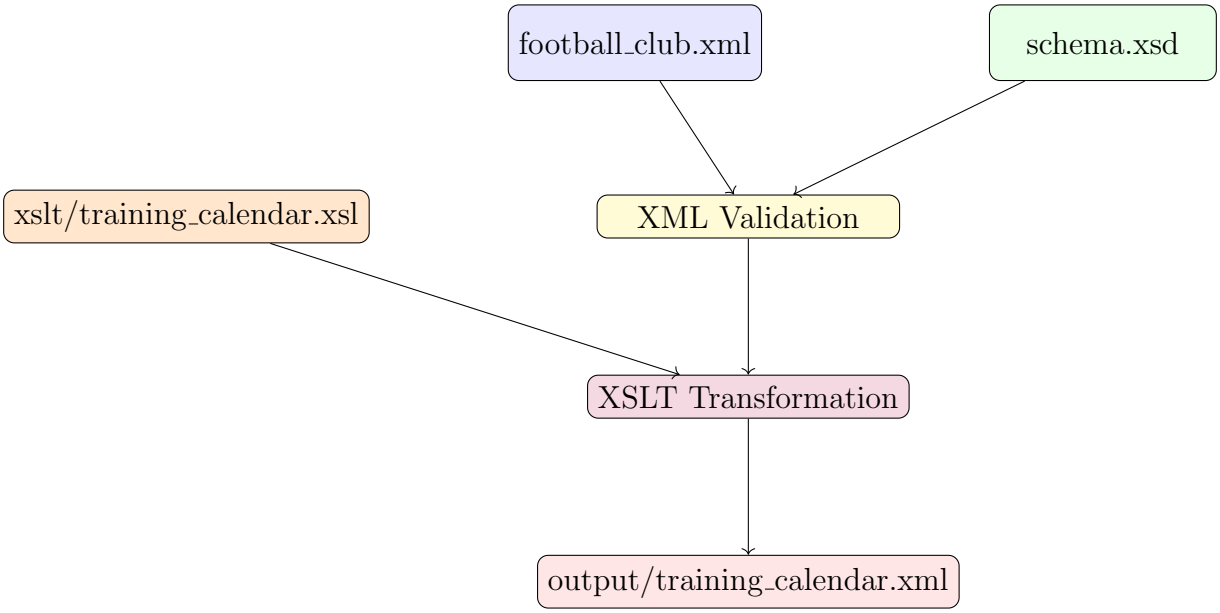


Figure 2: Training Calendar Generation (XML → XML)

These scenarios demonstrate XML restructuring while preserving semantic information.

## 2.2 XML to HTML Scenarios

Six XML to HTML transformation scenarios were implemented to provide visual reports and dashboards from the football club dataset.

- **Players Roster:** complete player directory including personal information, statistics, playing position, jersey number, and team assignment.
- **Team Overview:** teams organized by age category with squad composition grouped by position.
- **Match Calendar:** competition schedule showing match dates, kickoff times, venues, opponents, and results.
- **Coaching Staff:** directory of coaches with specialties, experience, assigned teams, and contact details.
- **Training Schedule:** weekly training sessions organized by team, coach, facility, and training type.
- **Club Dashboard:** executive summary including top scorers, top assistants, facilities overview, team metrics, and recent matches.

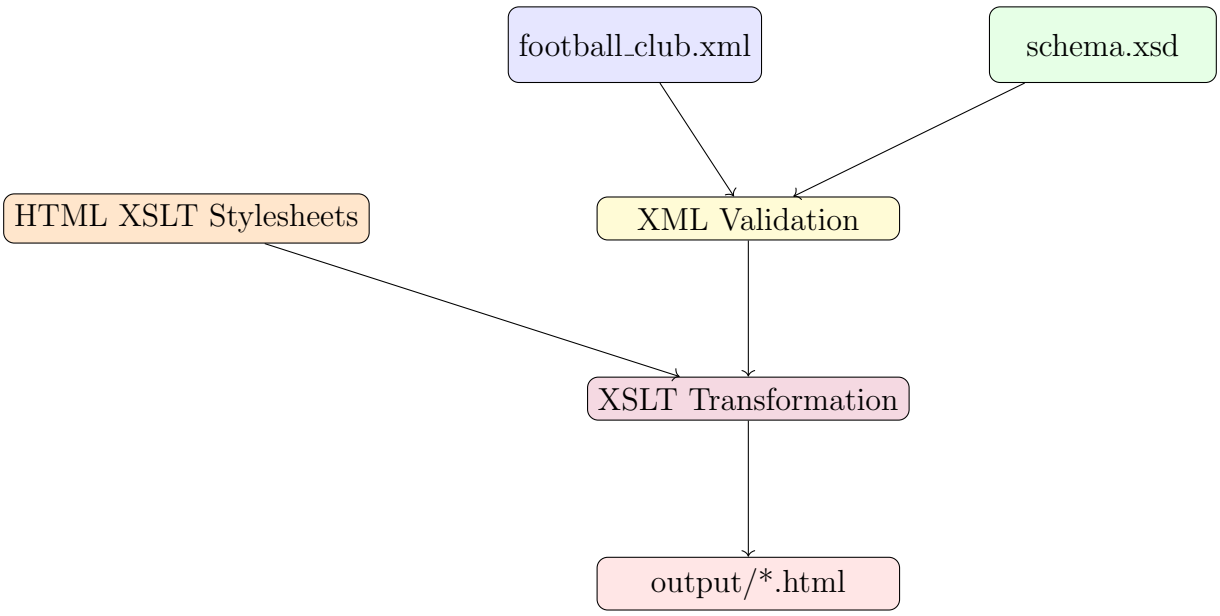


Figure 3: HTML Reporting Pipeline (XML → HTML)

These scenarios demonstrate how XML data can be transformed into user-friendly HTML reports for visualization and reporting purposes.

## 2.3 XML to JSON Scenarios

Two JSON transformation scenarios were implemented.

### Scenario 1: Active Members Export

This scenario extracts members with active memberships and exports them as JSON.

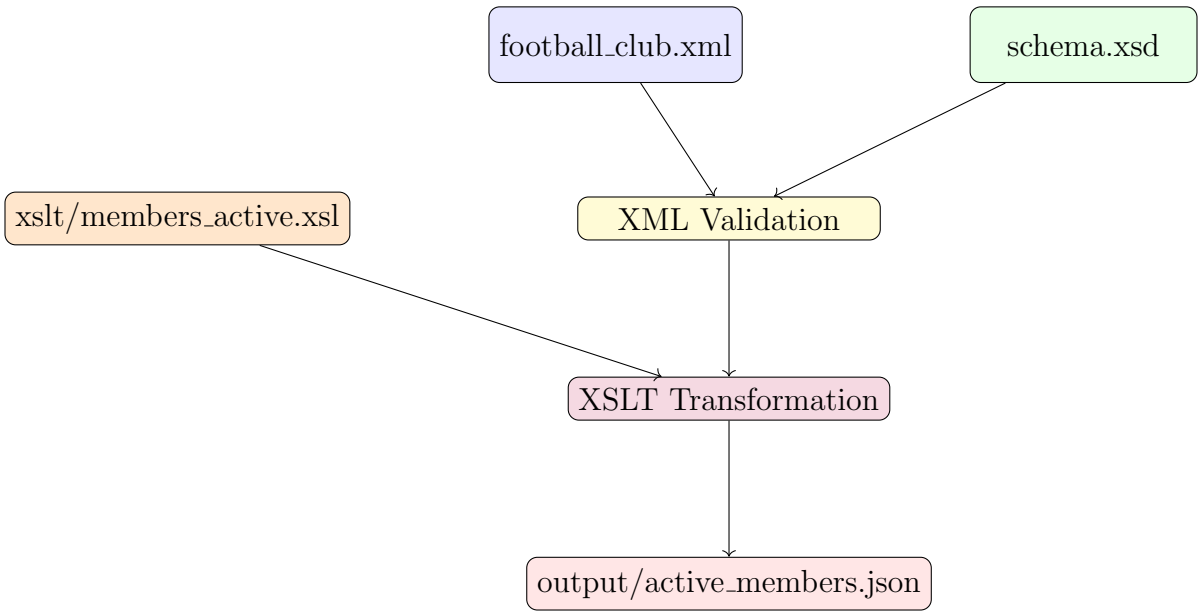


Figure 4: Active Members Export (XML → JSON)

### Scenario 2: Training Sessions by Coach

This scenario groups training sessions by assigned coach and exports the result in JSON format.

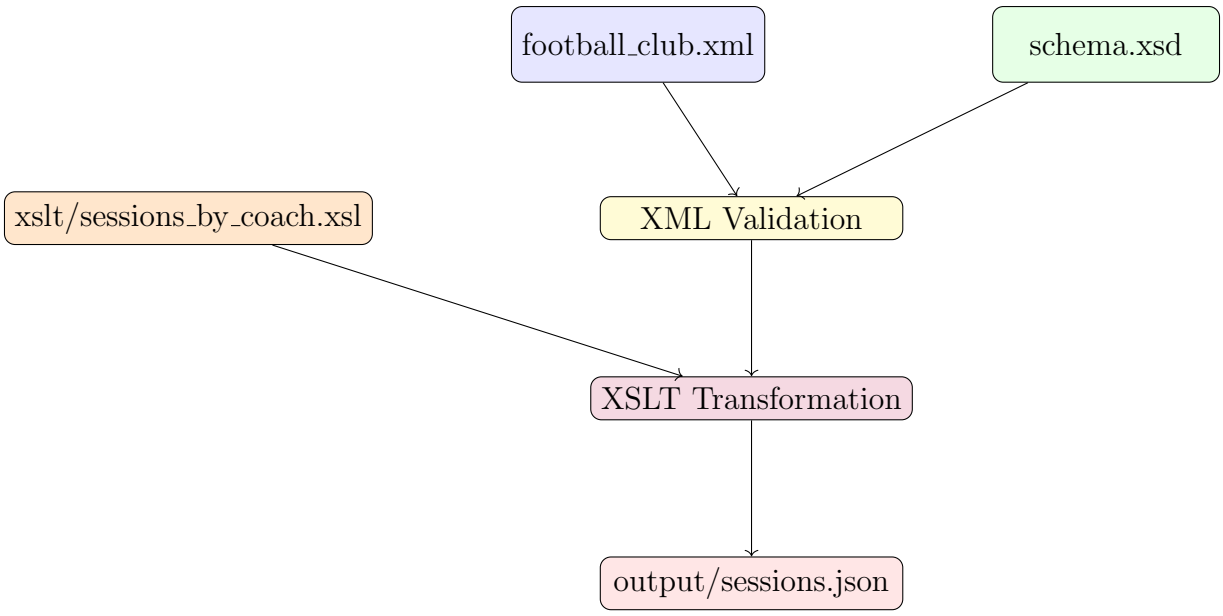


Figure 5: Training Sessions by Coach (XML → JSON)

These scenarios demonstrate how structured XML data can be transformed into lightweight and interoperable formats suitable for web applications, APIs, and data exchange.

# 3 Working Environment and Tools

The project was developed collaboratively using a combination of local development environments and online tools:

- **XML/XSD/XSLT Development:** Visual Studio Code with XML extensions for syntax highlighting and validation support.
- **XML Validation:** Python `lxml` library for programmatic validation of XML against XSD, and Java `javax.xml` APIs for the optional Java pipeline.
- **XSLT Processing:** Python `lxml.etree.XSLT` for applying stylesheets, and Java `javax.xml.transf` for the Java implementation.
- **Version Control:** GitHub (private repository) for collaborative file sharing and version tracking.
- **Report Writing:** Overleaf (online LaTeX editor) for collaborative report editing.
- **Communication:** Microsoft Teams for team coordination and file exchange.
- **Operating Systems:** Windows 10/11 across all team members.

# 4 Modeling Choices

## 4.1 Schema Design Principles

The XML Schema was designed following a **modular approach** with clear separation of concerns. The schema defines over 25 simple types with fine-grained restrictions and approximately 15 reusable complex types organized through inheritance.

### Simple Types with Restrictions

Rather than using generic `xs:string` or `xs:integer` types, we defined constrained simple types to enforce data quality at the schema level:

- **Pattern-based types:** The `idType` uses the pattern `[A-Z]{3}-[0-9]{3}` (e.g., PLY-001, COA-002) ensuring consistent identifiers across all entities. The `emailType` and `phoneType` use regex patterns for format validation.
- **Enumeration types:** Domain-specific values are constrained through enumerations. For example, `positionType` restricts player positions to {Goalkeeper, Defender, Midfielder, Forward}, and `matchResultType` to {Win, Loss, Draw, Pending}.
- **Range-restricted types:** Numeric fields use `minInclusive`/`maxInclusive` constraints. For instance, `jerseyNumberType` is restricted to 1–99, `minuteType` to 1–130 (accounting for extra time), and `percentageType` to 0–100 with one decimal digit.

### Complex Type Inheritance

A key design decision was using **type extension** through `xs:complexContent` and `xs:extension`. We defined a base `personType` containing shared attributes (name, date of birth, gender, nationality, contact), then extended it into specialized types:

- `playerType` extends `personType` by adding position, jersey number, preferred foot, physical measurements, and season statistics.
- `coachType` extends `personType` by adding specialty, license level, and years of experience.

This avoids redundancy and ensures consistency: any change to shared fields (e.g., adding an address to all persons) only requires modifying the base type.

### Referential Integrity with Keys and Keyrefs

To maintain data consistency across entities, we implemented **xs:key** and **xs:keyref** constraints at the root element level. For example:

- Each player's `teamRef` attribute must reference an existing team ID (`playerTeamRef` keyref).
- Each training session references valid team, coach, and facility IDs.
- Booking records reference existing facilities and coaches.

This ensures that the XML database cannot contain orphan references (e.g., a player assigned to a non-existent team).

## 4.2 A Modeling Problem: Match Event Modeling

One significant modeling challenge was representing **match events** (goals and cards) while maintaining traceability to individual players.

**The problem:** Each match can have multiple goals and cards. Each goal must reference the scorer (mandatory) and optionally an assist provider. Each card must reference the player and include a reason. These events must link back to players defined elsewhere in the document.

**Our solution:** We created dedicated complex types (`goalEventType` and `cardEventType`) that use **attributes for player references** and **child elements for event details**. The `scorerRef` attribute on a goal points to a player ID, while the minute and goal type are child elements. This hybrid approach (attributes for references, elements for data) follows XML best practices and keeps the structure readable.

**Advantage:** This design allows rich match reporting with full traceability. We can query across the dataset using XPath to compute statistics like "top scorers across all competitions" or "players with the most yellow cards."

**Trade-off:** We chose to embed match events inside their parent competition/match hierarchy rather than in a flat event list. This makes competition-level queries natural but cross-competition aggregation requires more complex XPath expressions. We accepted this trade-off because most real-world queries are competition-scoped.

## 4.3 Namespace Strategy

All elements use the namespace `http://www.dsti.football-club.com` with the prefix `fc:`. This avoids naming collisions and is consistent across the XML source, XSD schema, and all XSLT stylesheets, which must declare this namespace to correctly select elements.

# 5 Conclusion

This project successfully demonstrates the design and implementation of a complete XML data processing pipeline for a football club management system.

Starting from a structured XML source document and a fully defined XML Schema (XSD), the project ensured data consistency, integrity, and validation before applying multiple transformations.

Several transformation scenarios were implemented to illustrate different XML processing use cases:

- XML to HTML transformations for visualization and reporting
- XML to XML transformations for data restructuring
- XML to JSON transformations for lightweight data export and interoperability

In addition, Python and Java scripts were used to automate XML validation and XSLT execution, ensuring reproducibility and correctness of generated outputs.

This project highlights the practical use of XML technologies including:

- XML Schema Definition (XSD)
- XSLT transformations
- XPath-based data extraction
- Automated validation workflows

Overall, the project provides a complete demonstration of data modeling, transformation, validation, and output generation within a real-world structured data pipeline.